MAE 3780 Final Report

Group #48
Joseph Dalton, Manuely Feliz Portes, Kevin Sturm
Jmd546, mf833, kss247
May 12 2026

Robot Design and Strategy Review

The following is our final robot design:

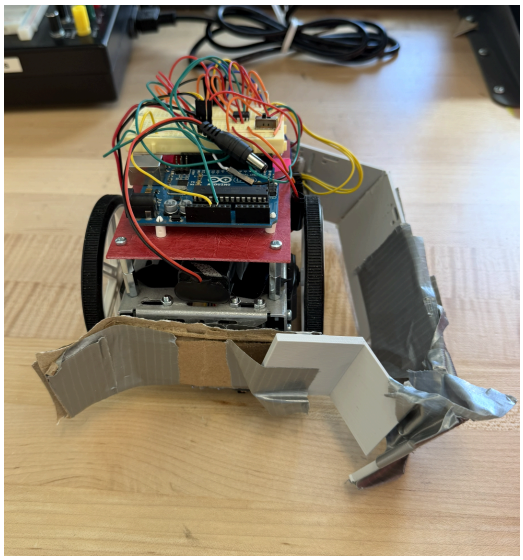

Figure 1: *Final Design for our Robot Front View*

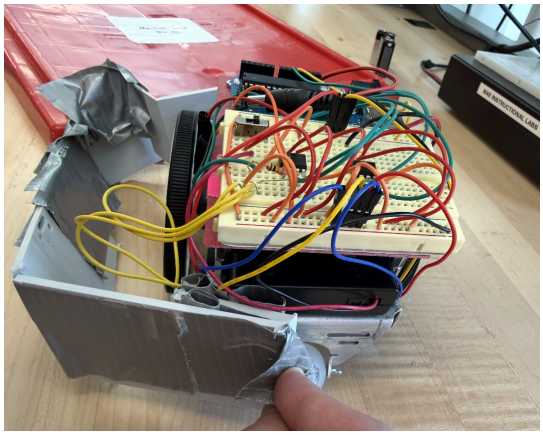

Figure 2: *FInal Design for our Robot Side View*

Mechanical Design

At the start of the project, we planned to use a servo-actuated collection mechanism with two arms that would open outward to create a larger collection area for blocks. Each side of the mechanism consisted of a servo connected to a custom 3D-printed arm. We designed and CADed mounting components that would allow the arms to attach securely to the servos and to the front of the robot. However, after printing the parts, we discovered several scaling issues in the CAD model. The mounting holes on the front attachment points did not align correctly, and the openings intended for the servos were undersized. As a result, the parts could not be assembled or used as intended.

To adapt, we shifted to a simpler and more reliable design using cardboard, spare 3D-printed pieces, and duct tape to construct a fixed arm capable of hooking and collecting blocks. The cardboard and duct tape proved to be cost effective, as they became critical components for collecting blocks while being free at the lab. While this design was less advanced than our original concept, it eliminated the need to troubleshoot servo reliability and significantly simplified both construction and programming. The simplified design also supported a streamlined driving strategy in which the robot only turned to the right, helping keep collected blocks contained within the collection area. Ultimately, the revised mechanism was easier to build, more dependable, and required no additional prototyping.

Electrical Design

Our electrical system was intentionally simple and focused on reliability. Two H-bridges were used to independently control the drive motors for each wheel. A physical switch was included to start and stop the drive motors by opening or closing the circuit.

Power was supplied using two separate sources. A 4xAA battery pack powered the drive motors, while a 9V battery powered the Arduino microcontroller. The Arduino also supplied power to the QTI sensor used for boundary detection and to the color sensor used for midline detection in milestone 4. A more detailed representation of the electrical layout can be found in Appendix B.

Software Design

The software underwent the largest number of revisions throughout the project. Initially, we planned to incorporate an ultrasonic sensor that would detect nearby opponents and trigger the robot to ram them. After further discussion and testing, we decided to abandon this idea in favor of a simpler and more reliable strategy.

Our final algorithm focused on predictable functionality rather than aggressive behavior. The robot continuously drove forward until the QTI sensor detected the arena boundary. When the border was detected, the robot turned to the right by a randomized angle between approximately 90 and 230 degrees before continuing forward. This approach was intended to allow the robot to repeatedly pass through the center of the arena to collect blocks while maintaining a less predictable movement pattern. In order to make the robot turn a specified amount of degrees, the following formula was used to convert the angle to a time in which the wheels would spin in opposite directions: $\frac{\theta \cdot \pi \cdot d}{360 \cdot \omega \cdot 3.5}$, where d is the track width of the robot and ω is the rotational speed of the motors.

The program did not account for which side of the arena the robot occupied, nor did it react differently when crossing between sides. Although simple, the algorithm was consistent and easy to implement, which aligned with our overall design philosophy.

Design Process Reflection

As discussed previously, our strategy changed significantly before Milestone 4. Our original design relied heavily on servo-controlled arms mounted to the front of the robot. However, once the parts arrived and were tested, we found that the servos were fragile and improperly sized for our printed components. This forced us to reconsider our approach and ultimately led us toward a simpler and more practical design.

Our milestone progression was fairly linear. We initially focused on building a functioning robot capable of completing Milestone 2, then gradually added additional features and improvements over time. While we had a general concept for the final robot early in the process, we treated each milestone as an independent objective and developed the robot incrementally.

Aside from the issues with the original 3D-printed servo mechanism, we did not encounter any major setbacks during development.

Competition Analysis

Our performance was as follows:

Wins	Losses	Draw
2	3	2

On competition day, our robot did not perform as well as we had hoped. During matches, the robot drove off the arena once and was pushed off multiple times by opposing robots. Additionally, several blocks were lost from the collection area because the arena surface was elevated above the surrounding floor, causing blocks to fall out when the robot crossed the boundary.

After the robot drove off the arena, we inspected it and discovered that one of the wires that connected the QTI sensor to the arduino had become disconnected. Aside from this hardware issue, the robot's software behaved as intended throughout the competition. Most of our difficulties stemmed from interactions with opposing robots, which we had not thoroughly tested during development.

Another contributing factor was that the practice arenas available in the lab were not exact replicas of the official competition arena. Specifically, the elevation of the competition platform was not present in the lab, so it was hard to account for come competition day. Because of these differences, we were unable to identify certain failure modes before competition day.

Despite these challenges, we were satisfied with many aspects of our performance. The systems that were within our control generally operated as expected. With additional testing and redesign, we believe we could improve the robot's resistance to becoming stuck and incorporate a floor or retaining mechanism to prevent blocks from falling out during boundary crossings.

Conclusions

If we were to repeat this project, we would prioritize simplicity from the beginning. Our original servo-based collection system required significantly more testing, fabrication accuracy, and debugging than we realistically had time to support. Once we considered our schedule constraints and other project commitments, it became clear that a simpler and more robust mechanism was the better option.

Budget was not a major limitation during the project. Our primary purchases were a 3D-printed component and larger wheels. The larger wheels proved to be a successful design choice and improved the robot's mobility. If we were to redesign the robot, we would likely replace the improvised cardboard-and-duct-tape arm with a properly designed and tested 3D-printed arm while still maintaining an overall simple design philosophy.

Our primary advice for future teams is to keep the design as simple as possible. More complex systems can be interesting and ambitious, but they often introduce additional points of failure and require significantly more testing and iteration. A simpler design is generally more reliable and easier to refine within the project timeline.

Appendix A

Bill of Materials (BOM)						Total Cost:	$25.19	
						Shipping Total: $0.00	Non-Spare Cost: $25.19	Spare Cost: $0.00
Part Name with Link	Description	Supplier	Qty Non-Spare	Qty Spare	Unit Cost	Shipping / Additional Cost	Full Cost Non-Spare (Qty NS * Unit Cost)	Full Cost Spare (Qty Spare * Unit Cost)
Pololu Wheel for Standard Servo Spline	Bigger wheels for bot	Pololu	1	0	$9.49	$0.00	$9.49	$0.00
3D printed Contraption	enclosure for our robot to hold cubes	RPL	1	0	$15.70	$0.00	$15.70	$0.00

Figure 3: *Team BOM*

The final cost was $25.19.

Appendix B

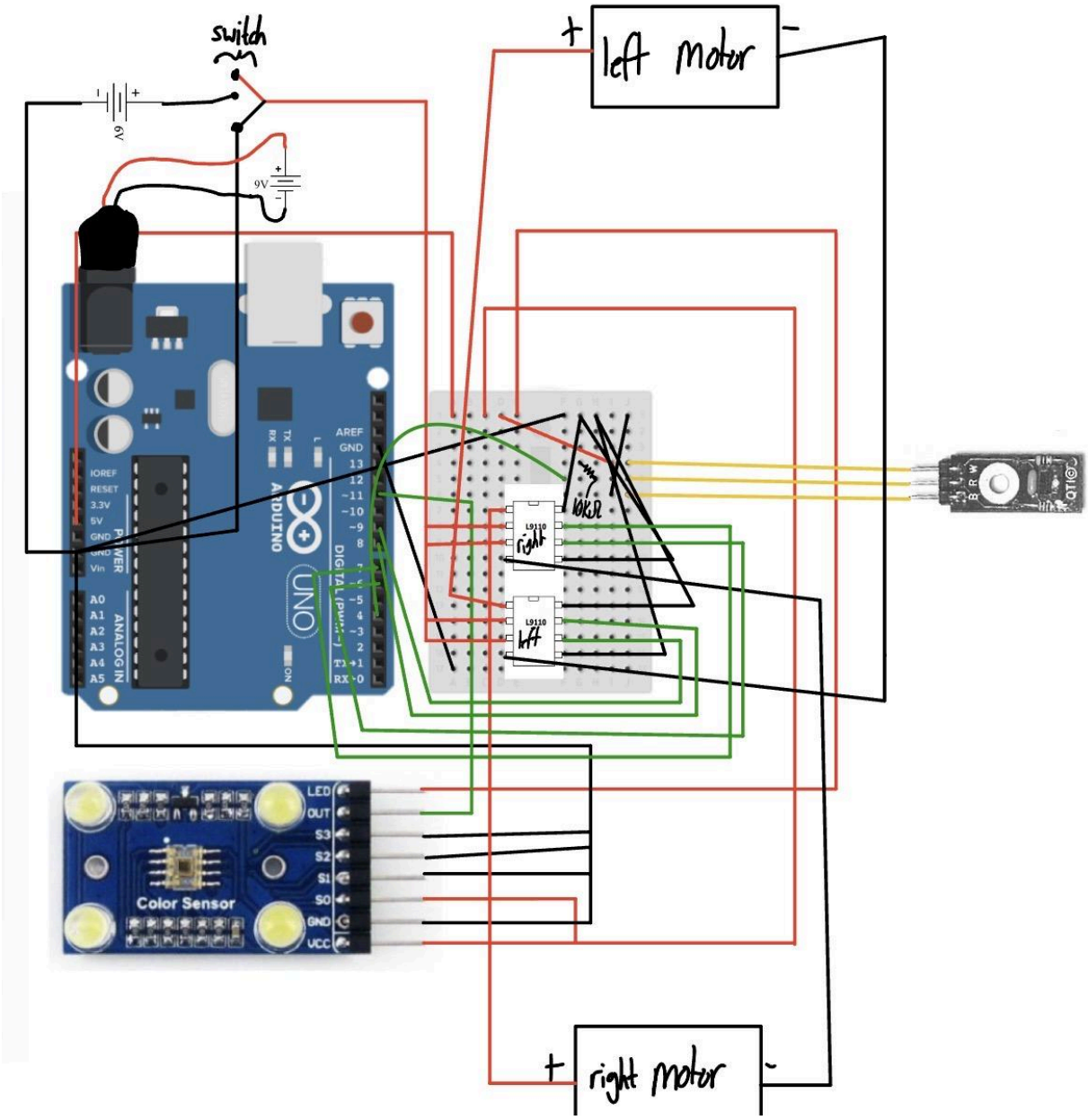

Figure 4: *Circuit diagram, notice common ground for all circuit elements except for 9V power source for arduino*

The wiring in this circuit diagram was drawn by hand in notability, using the prelab 4 image as a template to include the color sensor, and images of h-bridges, QTI sensors, and other circuit elements taken from various powerpoints and datasheets found in the canvas modules.

Appendix C:

The following outlines the CAD designs of the robot:

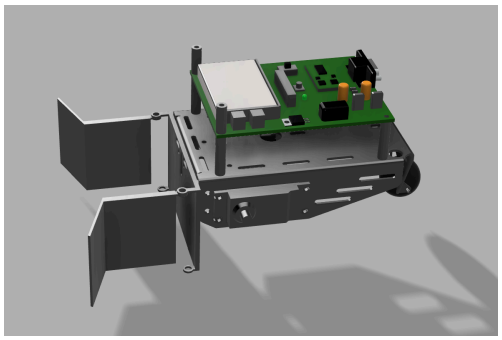

Figure 5: *Robot Ortho View*

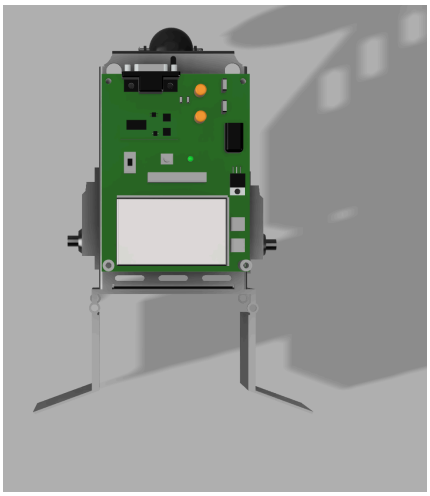

Figure 6: *Robot Top View*

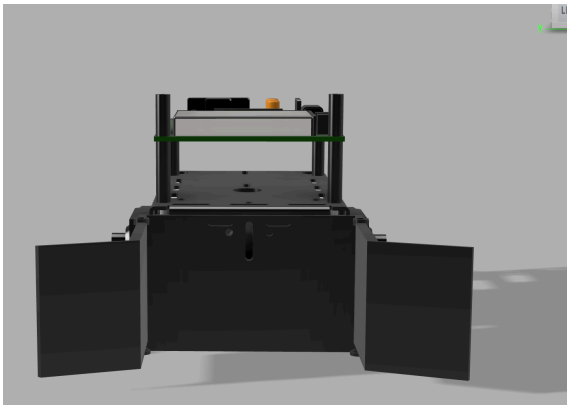

Figure 7: *Robot Front View*

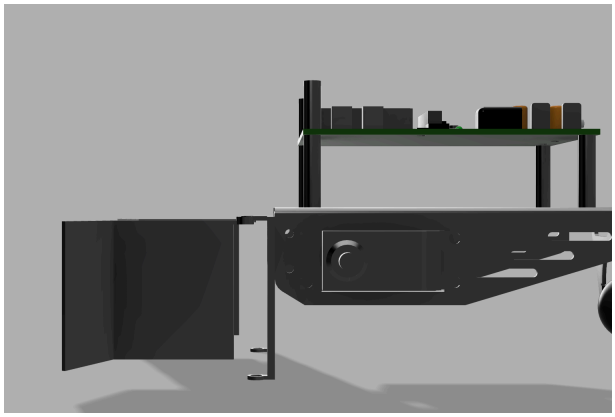

Figure 8: *Robot side view*

The original idea and iteration was for the robot's arms to close to protect the cubes when turning and from other robots.

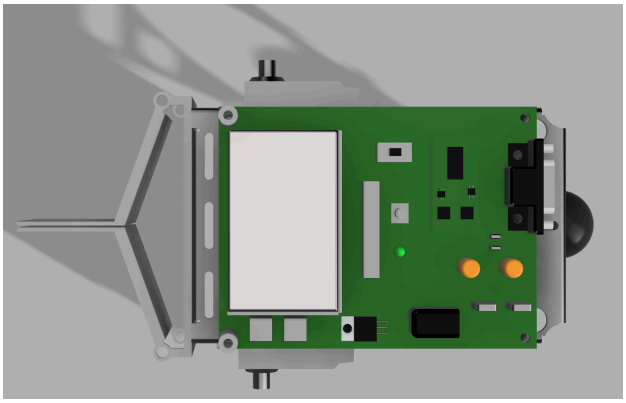

Figure 9: *Robot Arms closed top view*

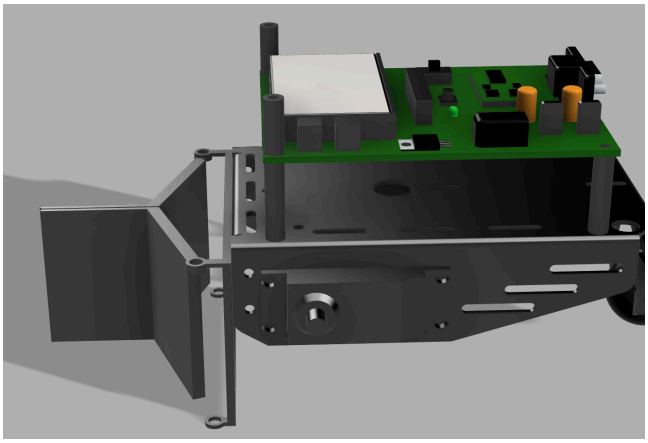

Figure 10: *Robot Arms Closed Ortho View*

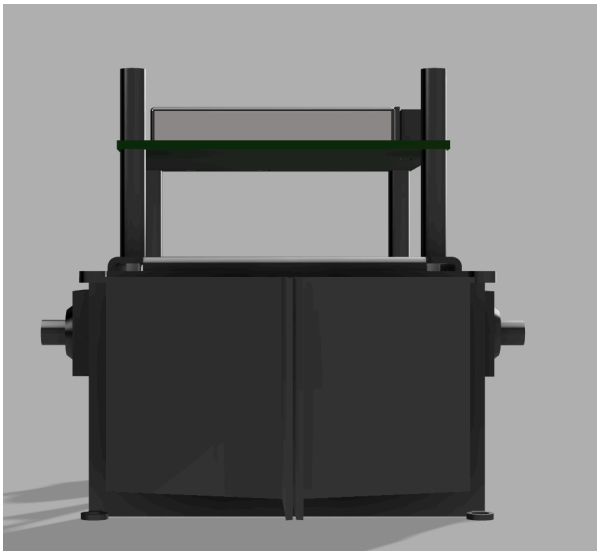

Figure 11: *Robot Arms Closed Front View*

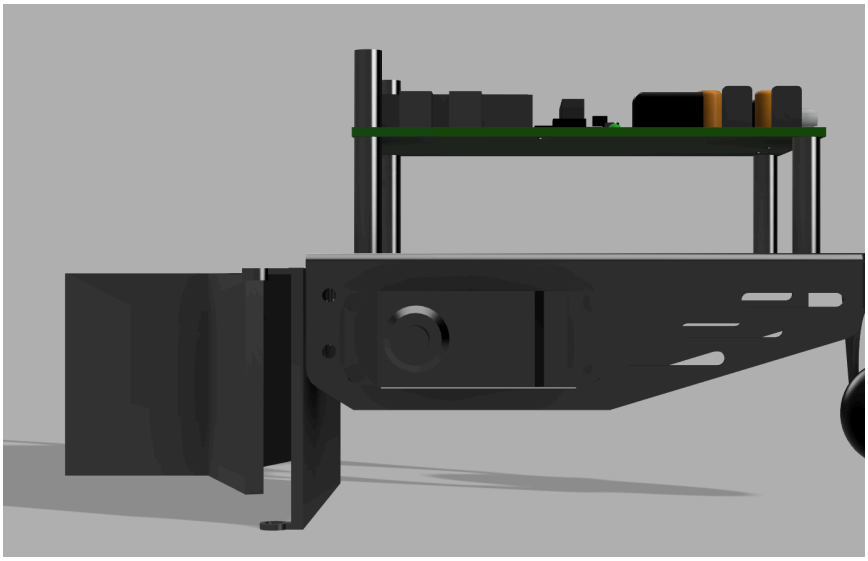

Figure 12: *Robot Arms Closed Side View*

The following is our first iteration of our arms. It got scratched because the cubes would potentially get under the robot and interfere with our sensors.

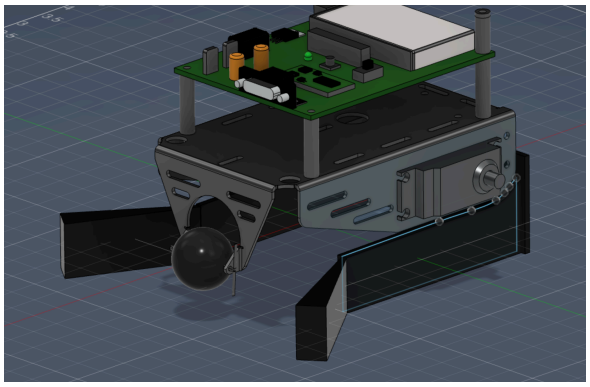

Figure 13: *1st Iteration*

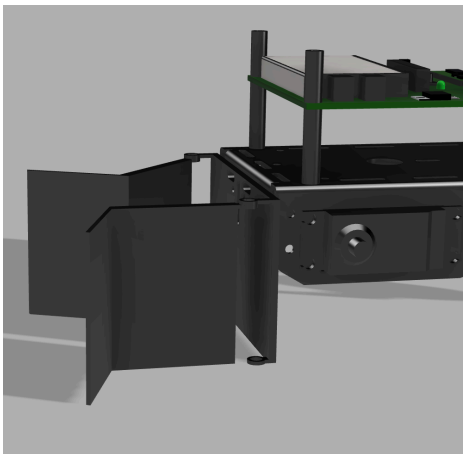

Figure 14: *2nd Iteration With Longer Arms*

We iterated and changed our arm length to be 0.75 from the floor in order to save weight and to ensure that our arms did not get caught on anything. Below are our arm measurements:

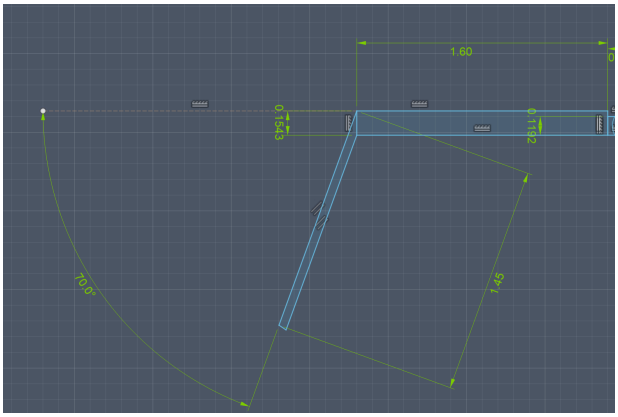

Figure 15: *Arm measurement dimensions.*

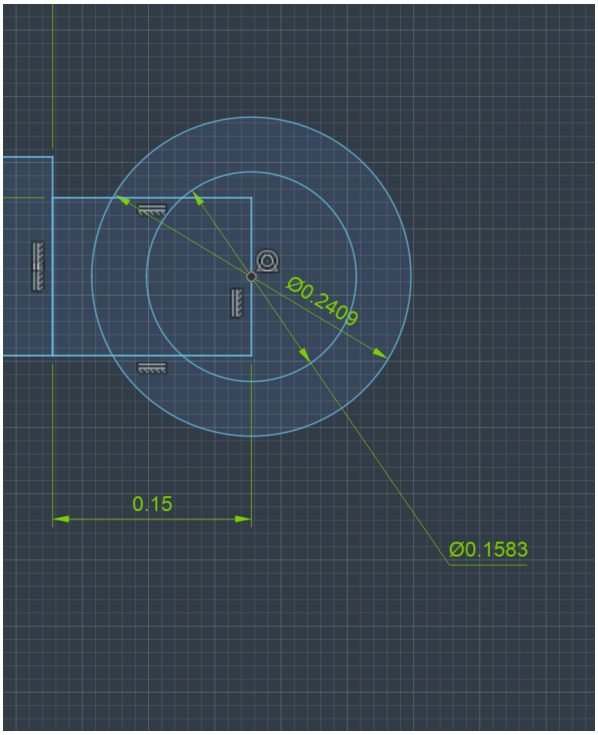

Figure 16: *Measurement of arm mounting point to back wall*

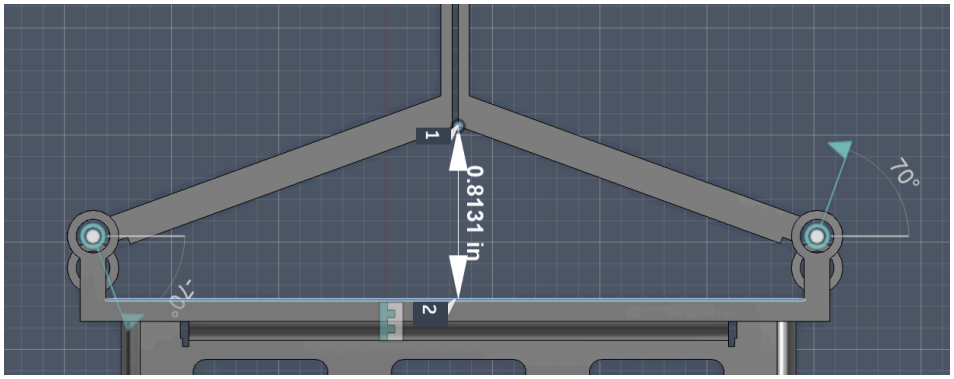

Figure 17: *Longest width when arms are closed to the wall*

This is when we realized that our arm closing strategy wouldn't have worked. Since these dimensions were the maximum we could have while still keeping within the 8x8 size constraints of the competition this had to be the size of the arms. We also would have never been able to close our arms completely which changed our strategy. Below you can find the mass and volume for our final 3D printed design.

Figure 18: *Volume of back piece*

Figure 19: *Volume of arm 1*

Figure 20: *Volume of arm 2*

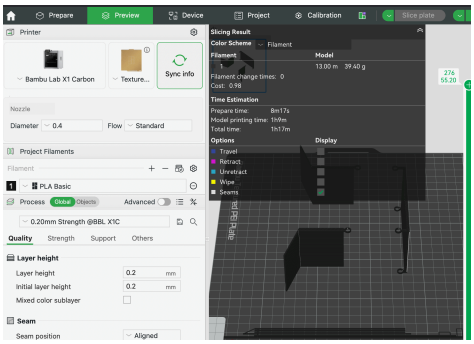

Figure 21: *Slice of our parts printed in PLA to find the weight. The total weight is 39.40g.*

Appendix D

When we first started brainstorming, we planned on adopting a strategy of starting at the edge of the board, turning at the middle, and stopping at the black line after all our blocks had been collected. Over time, we realized this wasn't the best strategy, as other teams could easily interfere with this plan if we weren't fast enough. The final strategy we settled on was just driving forward until we hit the black border, and turning a random amount of degrees (final range was between 90 and 230). This was a very simple yet effective strategy, as sometimes avoiding complexity can help to benefit reliability.

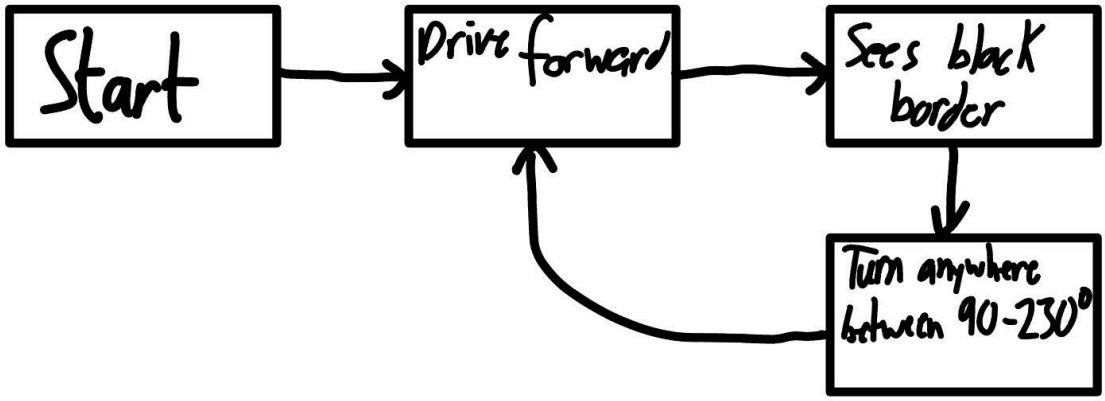

Figure 22: *Flowchart showcasing desired robot movement*

Appendix E

In our code there are some functions that go unused. These were leftovers from previous milestones that didn't make it into our competition code. The code for our robot is as follows

```
volatile int period = 0;
// "timer" : stores the value of TIMER1
volatile int timer = 0;
volatile int lastState = 0;
volatile int measurement = 0;
// --- ISR: an interrupt function that resets and reads the timer value

ISR(PCINT0_vect)
{
 static int edgeCount = 0;  // track how many rising edges we've seen

 if ((PINB & 0b00001000) && !(lastState)) {
   // Rising edge
   if (edgeCount == 0) {
     // First rising edge after re-enable: just start the timer, don't trust yet
     TCNT1 = 0;
     edgeCount = 1;
   } else {
     // Second rising edge: this is a real full period
     TCNT1 = 0;
   }
 }
 else if (!(PINB & 0b00001000) && (lastState)) {
   // Falling edge: only store if we've seen at least one clean rising edge
   if (edgeCount >= 1) {
     timer = TCNT1;
     period = timer;
   }
 }

 lastState = (PINB & 0b00001000);
}


// --- initColor: initializes the interrupt and timer

void initColor()
{

 // a. Initialize your I/O pins
 DDRB &= ~0b11110111; //pin 11 as input
 DDRD |= 0b00010000; //pin 4 as output
 // b. Initialize my pin change interrupt
 PCICR |= 0b00000001;
```

```
 PCMSK0 |= 0b00000000; //all initially disabled, will set up later
 // c. Initialize your timer
 TCCR1A = 0b00000000; //normal mode
 TCCR1B = 0b00000001; //prescalar of 1

 sei(); //enable all interrupts
}

// --- getColor: calculates the period of the sensor output wave and returns it as a
variable

int getColor() //return 0 for yellow, 1 for blue, 2 for red
{
 // Reset stale state before enabling
 TCNT1 = 0;
 period = 0;

 PCICR  |= 0b00000001;
 PCMSK0 |= 0b00001000;

 _delay_ms(50);  // long enough to capture at least 1 full period

 PCMSK0 &= ~0b00001000;
 PCICR  &= ~0b00000001;

 int hPeriod = period / 16;
 return (2 * hPeriod);
}

bool compareColor(){
 static int lastColor = 0;  // persists across calls
 static int colorName; //0 for yellow, 1 for blue
 int currentColor = getColor();
 if (currentColor > 25 && currentColor < 45){ // color is yellow
   colorName = 0;
 }
 else if (currentColor > 260 && currentColor < 310){ // color is blue
   colorName = 1;
 }
  if(lastColor != colorName){
   lastColor = colorName;
   return true;
 }
 else{
   lastColor = colorName;
   return false;
 }
 /*
 int diff = abs(currentColor - lastColor);
 if (diff > 20 && diff < 350){
   lastColor = currentColor;
   return true;
```

```
 }
 else{
   lastColor = currentColor;
   return false;
 }
 */
 _delay_ms(1000);
}

volatile int blackLineDetected = 0;
volatile int lastState2 = 0;

ISR(PCINT2_vect)
{
   // read full Port D
   int currentState = PIND & 0b00010000; // isolate pin 4

   // rising edge: 0 → 1 (black line detected)
   if (currentState == 0b00010000){
       blackLineDetected = 1;
   }
   else{
       blackLineDetected = 0;
   }
}

void initQTI()
{
   // set pin 4 (PD4) as input
   DDRD &= 0b11101111;

   // enable pin change interrupt for Port D group
   PCICR |= 0b00000100;   // PCIE2 = enable PCINT[23:16]

   // enable interrupt specifically for pin 4 (PCINT20)
   PCMSK2 |= 0b00010000;

   // initialize state
   lastState2 = PIND & 0b00010000;

   sei(); // global interrupt enable
}

volatile unsigned int servo_pulse_ticks = 100; // 1ms default
volatile unsigned int servo_tick_count = 0;

ISR(TIMER2_COMPA_vect) {
   servo_tick_count++;

   // start of 20 ms frame
   if (servo_tick_count == 1) {
       PORTD |= 0b00101000;   // P3 & P5 HIGH
```

```
    }
    // end pulse
    if (servo_tick_count == servo_pulse_ticks) {
        PORTD &= 0b11010111;   // P3 & P5 LOW
    }
    // reset every 20 ms (2000 * 10 µs)
    if (servo_tick_count >= 2000) {
        servo_tick_count = 0;
    }
}

void servo_timer2_init() {
    // CTC mode
    TCCR2A = 0b00000010;

    // prescaler = 8
    TCCR2B = 0b00000010;

    // 10 µs interrupt
    OCR2A = 0b00010011; // 19

    // enable compare match interrupt
    TIMSK2 |= 0b00000010;

    sei();
}

void servo_init() {
    // set P3 and P5 as output
    DDRD |= 0b00101000;

    servo_timer2_init();
}

void servo_set_angle(unsigned int angle) {
    if (angle > 70) {
        angle = 70;
    }
    // 1ms = 100 ticks, 2ms = 200 ticks
    servo_pulse_ticks = 100 + ((angle * 100) / 180);
}

volatile int omega = (int)4.7124;
void drive_forward_distance(int d){ //takes distance d in inches
    PORTB = (PORTB & 0b11111100) | 0b00000010;
  PORTD = (PORTD & 0b00111111) | 0b10000000;
  volatile float s = (d/(omega*3.5));
  volatile int t = (int) 700*s; //converts distance to a time
  Serial.println(t);
  for(int i = 0; i < t; i++){
   _delay_ms(1);
}         // wait for t milliseconds
```

```
}

void drive_forward(){ //drives forward continuously
   PORTB = (PORTB & 0b11111100) | 0b00000010;
  PORTD = (PORTD & 0b00111111) | 0b10000000;
}

void drive_backward_distance(int d){
   PORTB = (PORTB & 0b11111100) | 0b00000001;
  PORTD = (PORTD & 0b00111111) | 0b01000000;
  volatile float s = (d/(omega*3.5));
  volatile int t = (int) 750*s; //converts distance to a time
  Serial.println(t);
  for(int i = 0; i < t; i++){
   _delay_ms(1);
}         // wait for t milliseconds
}

void drive_backward(){
   PORTB = (PORTB & 0b11111100) | 0b00000001;
  PORTD = (PORTD & 0b00111111) | 0b01000000;
}

void drive_left_angle(int th){
   PORTB = (PORTB & 0b11111100) | 0b00000010;
  PORTD = (PORTD & 0b00111111) | 0b01000000;
  volatile float s = ((th*3.14159*1.6259845)/(360*omega*3.5));
  volatile int t = (int) 1800*s; //converts distance to a time
  Serial.println(t);
  for(int i = 0; i < t; i++){
   _delay_ms(1);
}         // wait for t milliseconds
}

void drive_left(){
   PORTB = (PORTB & 0b11111100) | 0b00000010;
  PORTD = (PORTD & 0b00111111) | 0b01000000;
}

void drive_right_angle(int th){
   PORTB = (PORTB & 0b11111100) | 0b00000001;
  PORTD = (PORTD & 0b00111111) | 0b10000000;
  volatile float s = ((th*3.14159*1.6259845)/(360*omega*3.5));
  volatile int t = (int) 1800*s; //converts distance to a time
  Serial.println(t);
  for(int i = 0; i < t; i++){
   _delay_ms(1);
}         // wait for t milliseconds
}

void drive_right(){
   PORTB = (PORTB & 0b11111100) | 0b00000001;
```

```
  PORTD = (PORTD & 0b00111111) | 0b10000000;
}

void stop_delay(int t){

  PORTB = (PORTB & 0b11111100) | 0b00000011;
  PORTD = (PORTD & 0b00111111) | 0b11000000;

  for(int i = 0; i < t; i++){
   _delay_ms(1);
}
}

void stop(){
  PORTB = (PORTB & 0b11111100) | 0b00000011;
  PORTD = (PORTD & 0b00111111) | 0b11000000;
}


//MAIN FUNCTION
int main(void){ // setup code that only runs once
 // set pins 8 and 9 as outputs for "left" motor
 initColor();
 initQTI();
 /*servo_init();
 servo_set_angle(0);
 servo_set_angle(70);*/
 DDRB |= 0b00000011; //set pins 8 and 9 as output for left motor
 DDRD |= 0b11000000; //set pins 6 and 7 as output for right motor
 Serial.begin(9600);
 while(1){ // code that loops forever
   if(blackLineDetected){

     drive_right_angle(random(90,230));
   }
   else{
     drive_forward();
/*
     //volatile int ignoreColor = 0;
     measurement = getColor();
     volatile int sensorEnabled = 1;
     if (compareColor()){
     sensorEnabled = 0;
     //stop_delay(500);

     PCMSK0 &= ~0b00001000;
     PCICR  &= ~0b00000001;

     drive_right_angle(90);
     drive_forward();
     bool ignore = compareColor();
     PCICR  |= 0b00000001;
```

```
     PCMSK0 |= 0b00001000;
*/
   }
}
   Serial.println(measurement);

 }
}
```